

ESTRUCTURAS DE DATOS

MARAIANA ACEVEDO GAÑAN

CEDULA:1035973358

BASES DE DATOS SQL Y NOSQL

FICHA:3169892

2025

- **¿Qué es un dato?**

Un dato es como una figura rara como un número, texto, foto, etc. de una cosa, suceso o movida. Solo así no tiene mucho sentido, pero al usarlo o juntarlo con otras cosas, puede soltar información buena. Los datos son lo más importante para guardar y usar cosas en las computadoras.

- **¿qué es información?**

La información es el resultado de procesar, organizar o estructurar datos. Aporta conocimiento, contexto y utilidad, permitiendo tomar decisiones. Nos da ideas, une cosas y sirve, para que podamos elegir bien. Como decir una lista de números sin más es solo datos, pero si esos números son ventas del mes, dan información útil a la empresa.

- **¿cómo se puede garantizar la disponibilidad de los datos en la empresa?**

La disponibilidad de los datos se puede garantizar mediante:

- Uso de sistemas de respaldo (backups).
- Implementación de servidores redundantes.
- Uso de bases de datos eficientes y con buen diseño.
- Políticas de mantenimiento y monitoreo constante.

Esto asegura que los datos estén accesibles cuando se necesiten, incluso ante fallos técnicos o desastres.

- **¿a qué nos referimos con confiabilidad de la información?**

La confiabilidad se refiere a que la información sea correcta, precisa y consistente. Esto implica que los datos no hayan sido alterados de forma indebida y que reflejen fielmente la realidad. Es esencial para que los análisis y decisiones basadas en estos datos sean válidas.

- **¿por qué consideras que la integridad de la información es esencial?**

La integridad asegura que los datos no hayan sido modificados de manera no autorizada o accidental. Si se pierde integridad, la información puede ser incorrecta, lo que puede llevar a errores graves en la toma de decisiones, pérdidas económicas o problemas legales.

• **¿qué tanto puede impactar una falla de autenticidad en los datos de una organización?**

Una falla de autenticidad significa que no se puede verificar el origen o la veracidad de los datos. Esto puede permitir:

- Manipulación o fraude.
- Pérdida de confianza por parte de clientes o socios.
- Decisiones incorrectas por parte de la empresa.
- En resumen, impacta seriamente la seguridad, reputación y funcionamiento de la organización.

Elabore una tabla en la cual simbolice las magnitudes siguientes en sistema numérico decimal y su correspondencia en los sistemas numéricos descritos:

Decimal	Binario	Octal	Hexadecimal
5050	100111011010	11632	13BA
4096	100000000000	10000	1000
4567	100011101111	10757	11D7
3800	111011011000	7330	ED8
1098	10001001010	2112	44A
1024	10000000000	2000	400
2048	100000000000	4000	800
2512	100111001000	4710	9D8
1970	11110111010	3732	7B2
680	1010101000	1250	2A8
512	1000000000	1000	200
300	100101100	454	12C
256	100000000	400	100
128	10000000	200	80
190	10111110	276	BE
64	1000000	100	40
56	111000	70	38
10	1010	12	A
780	1100001100	1404	30C

912	1110010000	1620	390
360	101101000	550	168
556	1000101100	1054	22C

428	110101100	654	1AC
990	1111101110	1756	3DE
604	1001011100	1114	25C
506	111111010	772	1FA
100	1100100	144	64

¿Qué operaciones aritméticas pueden ejecutarse con los sistemas numéricos?

En los sistemas numéricos, se pueden realizar diversas **operaciones aritméticas**, similares a las del sistema decimal, pero adaptadas a cada base. Algunas de las principales incluyen:

- **Suma:** Se realiza sumando dígitos en cada posición, teniendo en cuenta la base del sistema.
- **Resta:** Se efectúa restando dígitos con reglas específicas para cada base, utilizando el concepto de préstamo cuando es necesario.
- **Multiplicación:** Se aplica de manera similar a la decimal, pero ajustando los valores según la base. En binario, se usa la lógica de $0 \times 1 = 0$ y $1 \times 1 = 1$.
- **División:** Se ejecuta siguiendo el mismo principio de la división decimal, pero adaptada a la base correspondiente.

¿Cómo se hacen conversiones de un sistema a otro: binario a octal, octal a decimal, decimal a hexadecimal, etc.?

1. De Binario a Octal

El sistema octal está basado en el número 8, mientras que el binario se basa en 2. Para transformar un número binario a octal:

1. Se agrupan los bits en conjuntos de tres cifras, empezando desde la derecha.
2. Cada trío se convierte a su valor equivalente en octal.

Ejemplo: 101101010 (en binario)

Agrupamos: 101 - 101 - 010

Conversión:

- $101 \rightarrow 5$
- $101 \rightarrow 5$
- $010 \rightarrow 2$

Resultado en octal: 552

2. De Binario a Decimal

El sistema decimal utiliza la base 10 y el binario la base 2. Para hacer la conversión:

1. Cada cifra binaria se multiplica por 2 elevado a su posición (empezando por la derecha).

2. Se suman todos los resultados.

Ejemplo: 1101 (binario)

Cálculo:

$$(1 \times 2^3) + (1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0)$$

$$8 + 4 + 0 + 1 = 13$$

Resultado en decimal: 13

3. De Binario a Hexadecimal

El sistema hexadecimal trabaja con base 16. Para convertir desde binario:

1. Dividir el número binario en grupos de cuatro dígitos, desde la derecha.
2. Cada grupo se convierte en su valor hexadecimal correspondiente.

Ejemplo: 101101010

Agrupamos: 1011 - 0101 - 0 → agregamos un 0 a la izquierda para completar cuatro cifras: 0000

Grupos: 1011 - 0101 - 0000

Conversión:

- 1011 → B
- 0101 → 5
- 0000 → 0

Resultado en hexadecimal: B50

4. De Octal a Decimal

El sistema octal tiene como base el 8 y el decimal, el 10. Para realizar la conversión:

1. Cada cifra del número octal se multiplica por 8 elevado a su posición (desde la derecha).
2. Luego, se suman los resultados.

Ejemplo: 345 (octal)

Cálculo:

$$(3 \times 8^2) + (4 \times 8^1) + (5 \times 8^0)$$

$$192 + 32 + 5 = 229$$

Resultado en decimal: 229

5. De Octal a Hexadecimal

Para pasar de octal a hexadecimal, primero se convierte el número octal a binario, y luego ese binario se convierte a hexadecimal.

Ejemplo: 345 (octal)

1. Convertir cada cifra octal a su equivalente binario:

- $3 \rightarrow 011$
- $4 \rightarrow 100$
- $5 \rightarrow 101$

Binario resultante: 011100101

2. Agrupar en bloques de 4 bits: 0001 - 1001 - 0101

3. Convertir a hexadecimal:

- $0001 \rightarrow 1$
- $1001 \rightarrow 9$
- $0101 \rightarrow 5$

Resultado en hexadecimal: 195

6. De Decimal a Hexadecimal

Para transformar un número decimal a hexadecimal:

1. Dividir el número entre 16.
2. Anotar el residuo.
3. Repetir con el cociente hasta obtener 0.
4. El número hexadecimal se forma con los residuos en orden inverso (de abajo hacia arriba).

Ejemplo: 229 (decimal)

1. $229 \div 16 = 14$, residuo 5
2. $14 \div 16 = 0$, residuo E

Resultado en hexadecimal: E5

¿Qué son: bit, nibble, byte?

- Bit: Es la unidad más pequeña de información en computación. Representa un estado binario, es decir, 0 o 1.
- Nibble: Es un grupo de 4 bits. Se usa comúnmente en la representación de números en hexadecimal, ya que un nibble puede representar valores de 0 a 15.
- Byte: Es un conjunto de 8 bits. Es la unidad básica de almacenamiento en computadoras y permite representar 256 valores diferentes (de 0 a 255).

¿Cuál es la estructura de un registro de memoria?

Un **registro de memoria** es una unidad de almacenamiento dentro de un sistema informático que guarda datos temporalmente para su procesamiento. Su estructura varía

según el tipo de memoria y el propósito, pero generalmente incluye los siguientes componentes:

1. **Dirección de memoria:** Indica la ubicación específica del registro dentro del sistema de almacenamiento.
2. **Dato almacenado:** Contiene la información que se guarda en el registro, que puede ser un número, una instrucción o cualquier otro tipo de dato.
3. **Bits de control:** Incluyen señales que determinan el estado del registro, como permisos de lectura/escritura o indicadores de error.
4. **Tamaño del registro:** Define la cantidad de bits que puede almacenar (por ejemplo, 8, 16, 32 o 64 bits).
5. **Tiempo de acceso:** Indica la velocidad con la que se puede leer o escribir en el registro.

¿Cómo se crea un puntero en los lenguajes de programación?

Un **puntero** es una variable que almacena la dirección de memoria de otra variable. Se utilizan en lenguajes como **C**, **C++** y **Python** para manipular datos de manera eficiente.

Cómo crear un puntero en C/C++

En C y C++, los punteros se declaran con el operador `*` y se asignan con `&` (dirección de memoria):

```
#include <iostream>
using namespace std;

int main() {
    int num = 10; // Variable normal
    int *ptr = &num; // Puntero que almacena la dirección de num

    cout << "Valor de num: " << num << endl;
    cout << "Dirección de num: " << &num << endl;
    cout << "Valor almacenado en ptr: " << ptr << endl;
    cout << "Valor al que apunta ptr: " << *ptr << endl;

    return 0;
}
```

Aquí, `ptr` almacena la dirección de `num`, y `*ptr` accede al valor de `num`.

Cómo crear un puntero en Python

Python no tiene punteros explícitos como C/C++, pero usa referencias de objetos:

```
num = 10
ptr = num # ptr apunta a la misma referencia que num

print("Valor de num:", num)
print("Valor de ptr:", ptr)
```

Aquí, `ptr` no almacena una dirección de memoria directamente, pero apunta al mismo objeto que `num`.

¿Qué es un sistema de archivos y cuáles son los más utilizados?

Un **sistema de archivos** es el método que usa un sistema operativo para **organizar y administrar los datos** guardados en un dispositivo, como un disco duro, una memoria USB o una tarjeta SD.

Actúa como un **punto entre el sistema operativo y la información almacenada**, facilitando el acceso, lectura, escritura y gestión de archivos de manera ordenada y eficiente.

Principales sistemas de archivos

Según el sistema operativo y el tipo de almacenamiento, existen diferentes sistemas de archivos. Aquí algunos de los más populares:

- **FAT32:** Es uno de los más antiguos y ampliamente compatibles (funciona en Windows, macOS, Linux, consolas, etc.). Sin embargo, **no permite archivos mayores a 4 GB.**
- **exFAT:** Es la evolución de FAT32, sin las limitaciones de tamaño. Es perfecto para **memorias USB y discos externos**, ya que es compatible con varios sistemas operativos.
- **NTFS:** Sistema predeterminado en Windows. Soporta **archivos grandes, compresión de datos, permisos de seguridad y recuperación ante fallos.**
- **HFS+ / APFS:** Son los formatos usados por macOS. **HFS+** es más antiguo, mientras que **APFS** es el sistema moderno, optimizado para **rendimiento, seguridad y velocidad** en equipos Apple.
- **ext4:** Muy común en sistemas Linux. Es confiable, rápido y eficiente. Es la evolución de **ext2 y ext**

¿Cómo se almacenan los datos en un disco duro?

Los datos pueden almacenarse de diferentes formas según el tipo de disco: **HDD** o **SSD**.

1 Discos duros mecánicos (HDD)

- Funcionan con **platos giratorios** cubiertos con material magnético.
- Un **brazo mecánico** mueve un cabezal que **lee o escribe datos** en el disco.
- La información se divide en **pistas, sectores y cilindros.**
- La velocidad de acceso depende de las **revoluciones por minuto (RPM)** del disco.

Ejemplo: Son comunes en computadores de escritorio y portátiles antiguos.

2 Unidades de estado sólido (SSD)

- No tienen piezas móviles. Guardan la información en **chips de memoria flash NAND.**
- Permiten un acceso **mucho más rápido** a los datos.
- Son **más resistentes, silenciosos y eficientes** que los HDD.
- Ideales para **aumentar el rendimiento** de laptops, PCs y servidores.

Ejemplo: Al reemplazar un HDD por un SSD, una computadora puede arrancar en segundos.

¿Cuáles son los tipos de dato que existen en lenguajes de programación?

Los tipos de datos en programación son clasificaciones que indican qué clase de valores puede contener y manejar una variable. Se organizan por su naturaleza y finalidad. A continuación, se enumeran los más habituales:

1. Tipos de datos primitivos

Son los más elementales y están presentes en casi todos los lenguajes:

- **Enteros (int):** Números sin parte decimal, por ejemplo 10, -5, 1000.
- **Decimales (float, double):** Números con fracción decimal, como 3.14, -0.5, 2.718.
- **Booleanos (bool):** Solo permiten dos valores posibles: true (verdadero) o false (falso).
- **Carácter (char):** Representa un único símbolo, como 'A', '9', '%'.
• **Cadenas (string):** Conjunto de caracteres, como "Hola mundo".

2. Tipos de datos estructurados

Permiten contener varios valores bajo una sola variable:

- **Arreglos (array):** Colección de elementos del mismo tipo, por ejemplo [1, 2, 3].
- **Listas (list):** Similares a los arrays, pero con mayor versatilidad.
- **Diccionarios (dict, map):** Almacenan pares clave-valor, como {"nombre": "Ana", "edad": 30}.
- **Tuplas (tuple):** Secuencias inmutables, por ejemplo (2, "azul", False).

3. Tipos de datos abstractos

Estructuras utilizadas en programación orientada a objetos:

- **Clases y objetos (class):** Permiten crear tipos personalizados con atributos y métodos.
- **Enumeraciones (enum):** Definen un conjunto fijo de valores posibles, como Días = {Lunes, Martes, Miércoles}.

4. Tipos de datos especiales

Aparecen en ciertos lenguajes y casos particulares:

- **Nulo (null, None):** Representa que no hay valor asignado.
- **Punteros (pointer):** Guardan direcciones de memoria (frecuentes en C y C++).
- **Funciones (function):** En algunos lenguajes se pueden tratar como datos y asignar a variables (Python, JavaScript).

¿Cuáles son los tipos de dato que existen en las bases de datos?

Los sistemas de gestión de bases de datos permiten trabajar con distintos tipos de datos según la información que se requiera guardar. A continuación se agrupan los más comunes:

1. Datos numéricos

Se utilizan para valores con o sin decimales:

- **INT, BIGINT, SMALLINT:** Números enteros.
- **FLOAT, DOUBLE, DECIMAL:** Números con parte fraccionaria, ya sea de precisión variable o fija.

2. Datos de texto

Permiten almacenar caracteres y textos:

- **CHAR(n):** Cadena de longitud fija.
- **VARCHAR(n):** Cadena de longitud variable.
- **TEXT:** Texto largo sin una longitud máxima establecida.

3. Fechas y horas

Registran información temporal:

- **DATE:** Fecha solamente (formato YYYY-MM-DD).
- **TIME:** Solo hora (formato HH:MM:SS).
- **DATETIME / TIMESTAMP:** Combinan fecha y hora.

4. Booleanos

Representan valores binarios de verdadero o falso:

- **BOOLEAN:** Se codifica como 1 (TRUE) o 0 (FALSE).

5. Datos binarios

Usados para guardar archivos u otros datos en formato binario:

- **BLOB:** Grandes volúmenes de datos binarios.
- **BYTEA:** Tipo binario usado en PostgreSQL.

6. Datos espaciales

Aplicados en bases de datos que manejan información geográfica:

- **GEOMETRY:** Para almacenar datos espaciales como coordenadas.
- **POINT, LINESTRING, POLYGON:** Elementos geométricos específicos

¿Qué es un Collation en base de datos?

El **Collation** es un conjunto de reglas que determina cómo se comparan y organizan los caracteres dentro de una base de datos. Influye en aspectos como el orden alfabético, la distinción entre mayúsculas y minúsculas, y la consideración o no de acentos. Es clave para realizar búsquedas y ordenamientos que respeten el idioma o el contexto específico del texto almacenado.